

SPD Controller (SPDC)

Technical Design Documentation

Source Code Reference & Module Description

1. Project Overview

The SPD Controller (SPDC) is an RTL subsystem that reads Serial Presence Detect (SPD) EEPROM data from DDR3 DIMM modules over the I2C bus, stores it in three large register banks, and then decodes the raw SPD bytes into processed DDR3 timing parameters expressed in clock-cycle units (nCK) for consumption by the DDR3 Memory Controller (DDR3MC).

SPD EEPROMs are physically located on DDR3 DIMMs and contain JEDEC-standardized byte maps describing the DIMM's geometry, timing requirements, and electrical characteristics. The SPDC reads up to three DIMM slots sequentially over I2C, stores their 256-byte SPD images, detects which slots are populated (by checking for I2C acknowledgement), performs fixed-point arithmetic to convert raw SPD byte values into nCK timing values, and then streams the decoded results to the DDR3MC via a valid/address/data interface.

1.1 Key Characteristics

Parameter	Value	Notes
System Clock	200 MHz (5 ns period)	SPDC_CLK_PERIOD_IN_FIXED_POINT_FORMAT = 8'b000101000
Max DIMM Slots	3	DIMM0, DIMM1, DIMM2 (3x spdc_ctrl reg banks)
SPD Bytes Per DIMM	256	MAX_BYTE_COUNT = 256
SPD Storage Width	2048 bits per bank	32 x 64-bit registers (reg0/reg1/reg2)
I2C SCL Frequency	~390 kHz	Host clock / 2^CNTR_W countdown (CNTR_W=10 by default)
Decoded Output Width	32 bits (data), 16 bits (address)	SPDC_DEC_DATA_WIDTH=32, SPDC_DEC_ADDR_WIDTH=16
Commands Supported	CMD_NOP, CMD_SPD_WR, CMD_SPD_RD, CMD_DEC_RD	2-bit command bus
Fixed-Point Arithmetic	Q3/8-bit, Q3/12-bit, Q3/16-bit, Q5/13-bit	Multiple spdc_qdiv and spdc_qmults instances in spdc_dec

1.2 Supported SPD Bytes Decoded

The decoder processes the following JEDEC DDR3 SPD byte positions to compute final timing values in nCK units:

SPD Byte(s)	Parameter Decoded
Byte 2	Device Type (DDR3 identification)
Byte 3	Module Type (SODIMM identification)
Byte 4	SDRAM Capacity / Bank Address Bits

Byte 5	Row / Column Address Bits
Byte 7	Module Organization (ranks, device width)
Byte 8	Module Memory Bus Width (primary + extension)
Byte 9	Fine Timebase (FTB) dividend/divisor
Bytes 10-11	Medium Timebase (MTB) dividend/divisor
Bytes 12-13	tCK (min clock cycle time) + SDRAM speed grade
Bytes 14-15	CAS Latency (CL) supported values bitmap
Byte 16	tAA (CAS latency time in MTB units)
Byte 17	tWR (write recovery time)
Byte 18	tRCD (RAS-to-CAS delay)
Byte 19	tRRD (rank-to-rank delay)
Byte 20	tRP (row precharge time)
Bytes 21-22	tRAS (active-to-precharge time, 12-bit)
Bytes 21+23	tRC (active-to-active, 12-bit)
Bytes 24-25	tRFC (refresh-to-activate, 16-bit)
Byte 26	tWTR (write-to-read delay)
Byte 27	tRTP (read-to-precharge delay)
Bytes 28-29	tFAW (four-activate window, 12-bit)
Bytes 33	Upper nibbles for tRAS/tRC
Bytes 34-38	Fine Timebase correction offsets for tCK, tAA, tRCD, tRP, tRC
Derived (fixed)	tCCD, tZQINIT, tREFI, tburst, tMRD, tMOD, tXPR, tRTW, t500us, t200us, t10ns, t7.5ns

2. System Architecture

2.1 Block Diagram (Textual)

```
[ spdc_wrap ]  -- FPGA harness: MMCM clock, test driver, DUT
|
[ spdc_top ]   -- Top-level FSM: sequences SPD write, wait, decoder trigger
|
[ spdc_ctrl ]  -- Command dispatcher: I2C read, register storage, decode trigger
/              \
spdc_i2c_top    spdc_dec
/              \
spdc_i2c_st     spdc_i2c_clk |
(I2C master     (SCL clock   |
protocol        divider)    |
FSM)                  |
                      [fixed-point arithmetic]
                      spdc_qdiv (x7 instances)
                      spdc_qmults (x4 instances)
                      spdc_qadd (x2 instances)
                      spdc_pipeline (delay line for slv_write_en)
```

2.2 Operational Flow

On reset deassertion, `spdc_top` immediately begins the SPD-write sequence (`CMD_SPD_WR`), which causes `spdc_ctrl` to sequentially read all 256 bytes from each DIMM's I2C EEPROM and store them into `spd_reg0`, `spd_reg1`, and `spd_reg2` (one bank per DIMM slot). `spdc_ctrl` uses I2C slave address bits (`i2c_slv_addr` = 0, 1, 2) to select between the three DIMM slots.

After all three banks are filled, `spdc_top` waits for `dec_en` to be asserted (driven by `spdc_testDrv` in hardware or the external system). On `dec_en`, `spdc_top` issues `CMD_DEC_RD`, which causes `spdc_ctrl` to assert `dec_rd_en` to `spdc_dec`. The decoder then processes all stored SPD bytes, performs the fixed-point timing calculations, and streams 57 decoded parameters (address 0x0000 through 0x0037, plus bookend markers at 0xFFFF) to the DDR3MC config interface via `valid/data_out/address` outputs.

3. Source File Descriptions

This section documents every source file in the SPDC project, organized by folder.

include/

spdc_parameters.vh

Global parameter header included by all RTL and simulation files. Defines all shared constants across the SPDC project.

Parameter / Constant	Description
MAX_BYTE_COUNT = 256	Number of SPD bytes to read per DIMM device. Determines the depth of the I2C sequential read loop.
CNTR_W = 10	Width of the I2C SCL clock divider counter in spdc_i2c_clk. Produces a tick every $2^{10} = 1024$ host clock cycles, giving ~195 kHz SCL at 200 MHz host.
SPDC_CMD_WIDTH = 2	Width of the command bus between spdc_top and spdc_ctrl: CMD_NOP(0), CMD_SPD_WR(1), CMD_SPD_RD(2), CMD_DEC_RD(3).
SPDC_SPD_ADDR_WIDTH = 12	Width of the SPD register address (12 bits, supporting up to 4096 byte locations across 3 DIMMs x 256 bytes + page select).
SPDC_SPD_DATA_WIDTH = 64	Width of the SPD register read data output — 64 bits (8 bytes at a time) returned from spdc_ctrl to spdc_top on SPD_RD commands.
SPDC_DEC_DATA_WIDTH = 32	Width of decoded output data from spdc_dec to the DDR3MC config interface.
SPDC_DEC_ADDR_WIDTH = 16	Width of decoded output address from spdc_dec. Address 0xFFFF is used as a start/end marker; addresses 0x0000..0x0037 carry actual timing parameters.
SPDC_INTF_FIFO_WRITE/READ_DATA_WIDTH = 122/114	Widths of the interface FIFO (if present in a future integration path), currently unused in the active RTL path.
CMD_SPD_WR=1, CMD_SPD_RD=2, CMD_DEC_RD=3, CMD_NOP=0	Command encodings for the spdc_top → spdc_ctrl command interface.
SPDC_STATUS_WIDTH=8, STATUS_BIT_DEC_DONE=1	Status register width and the bit position of the decoder-done flag.

SPDC_DATAGRAM_CMDWRD_* fields	Bit-position macros for the command-word datagram packing format (used in a future or alternate FIFO path, not in active RTL).
CMDWRD_CMD_READ = 2'b01	Read command encoding for the datagram command word.
SPDC_CLK_PERIOD_IN_FIXED_POINT_FORMAT = 8'b000101000	The host clock period (5 ns = 200 MHz) encoded in fixed-point Q3/8 format (000101.000 binary), passed to spdc_dec to normalize all SPD timing byte values against the system clock.

spdc_wrap.sv

Top-level FPGA integration wrapper. Under the SYNTHS compile flag it exposes real FPGA pins (differential clock, I2C SDA/SCL/SA) and instantiates the MMCM clock generator (clkgen). In simulation mode (no SYNTHS), it wires internal signals directly. Instantiates: the MMCM IP (SYNTHS only), a reset synchronizer, spdc_top (the functional DUT), and spdc_testDrv (the test stimulus driver).

Sub-instance	Description
clkgen (SYNTHS only)	Xilinx MMCM generating the 200 MHz system clock from a differential input reference (gclk_p/gclk_n).
reset_sync (implied)	Two-flop reset synchronizer (locked && greset_n → synchronized reset_n for spdc_top domain).
spdc_top	Functional SPD controller top module.
spdc_testDrv	Test driver providing dec_en stimulus in both simulation and hardware (via VIO if SYNTHS).

spdc_top.v

Functional top-level FSM. Sequences the three main phases of SPDC operation: SPD read (issuing CMD_SPD_WR to trigger I2C reads of all DIMMs), waiting for an external decode enable (dec_en), and decoder read (issuing CMD_DEC_RD to trigger the fixed-point timing decode). Instantiates a single spdc_ctrl instance which it commands via a 2-bit cmd bus.

State Machine States

State	Description
IDLE	On reset deassertion, immediately transitions to SPD_WRITE_CMD with no wait.
SPD_WRITE_CMD	Asserts CMD_SPD_WR to spdc_ctrl for one cycle, then proceeds to wait for completion.
SPD_WRITE_CMD_WAIT	Waits for spdc_done (from spdc_ctrl) to confirm all three DIMM SPD banks have been read over I2C.
WAIT	Holds here until dec_en is asserted by spdc_testDrv. SPD random-read path (spd_rd_en) is present in the state machine but commented out as unused.
DEC_READ_CMD	Asserts CMD_DEC_RD to spdc_ctrl for one cycle.
DEC_READ_CMD_WAIT	Waits for spdc_done confirming decode is complete.
DONE	Returns to WAIT, ready to accept another dec_en trigger.

SPD_READ_CMD / SPD_READ_DATA_VALID	States for the SPD random-read path (currently disabled by commented-out condition in WAIT state).
CMD_ERR	Error state for invalid command (not reachable in current flow).

Key Port Connections

Port	Connection / Description
clk / reset_n	200 MHz system clock and active-low reset from spdc_wrap.
reg0/reg1/reg2 [2047:0]	Output register banks of raw SPD bytes (DIMM0/1/2). 2048 bits = 256 bytes per bank, passed straight through from spdc_ctrl.
dec_data_valid / dec_addr / dec_data	Decoded timing outputs passed from spdc_ctrl/spdc_dec to the DDR3MC config interface.
dec_en	Input from spdc_testDrv; triggers the CMD_DEC_RD sequence.
i2c_SDA / i2c_SCL	I2C bus signals, passed through to spdc_ctrl then to spdc_i2c_top.

spdc_ctrl.v

Central command dispatcher and register manager. Accepts 2-bit commands from spdc_top and performs the corresponding action: sequential I2C reads into one of three SPD register banks (CMD_SPD_WR), random SPD register reads (CMD_SPD_RD, disabled in current flow), or triggering the SPD decoder (CMD_DEC_RD). Instantiates both spdc_i2c_top and spdc_dec.

State Machine States

State	Description
IDLE	Waits for a non-NOP command. On CMD_SPD_WR → SPD_WR_REG0_EN. On CMD_SPD_RD → selects REG0/1/2 based on addr[11:8]. On CMD_DEC_RD → DEC_RD_EN.
SPD_WR_REG0_EN	Enables I2C with i2c_slv_addr=0 (DIMM0). Waits for i2c_done asserted (I2C read complete).
SPD_WR_REG0_DONE	Waits for i2c_done to deassert (I2C bus released) before moving to next DIMM.
SPD_WR_REG1_EN	Enables I2C with i2c_slv_addr=1 (DIMM1). Same done/deassert pattern.
SPD_WR_REG1_DONE	Waits for I2C bus release before DIMM2.
SPD_WR_REG2_EN	Enables I2C with i2c_slv_addr=2 (DIMM2). Same done/deassert pattern.
SPD_WR_REG2_DONE	All three DIMMs read. Asserts done to spdc_top.
SPD_RD_REG0/1/2_EN	Reads one 64-bit word from the appropriate spd_reg bank at addr[7:0]>>3. Asserts spd_data_valid for one cycle.
DEC_RD_EN	Asserts dec_rd_en to spdc_dec for one cycle. Waits for dec_rd_done (from spdc_dec). Then DONE.
DONE	Pulses done to spdc_top for one cycle, then returns to IDLE.

Register Storage Architecture

spdc_ctrl maintains three internal register arrays: spd_reg0[31:0], spd_reg1[31:0], spd_reg2[31:0] — each element 64 bits wide, totaling $32 \times 64 = 2048$ bits = 256 bytes per DIMM. During I2C reads, each byte received (i2c_data_read, 8 bits) is shifted into the appropriate register array entry at position $\text{byte_cntr} \gg 3$ (which DIMM's 64-bit word this byte belongs to), using a right-rotate shift: $\{\text{i2c_data_read}, \text{spd_reg0}[\text{byte_cntr} \gg 3][63:8]\}$. The completed arrays are concatenated as reg0/reg1/reg2 outputs via a generate loop: $\text{assign reg0}[(j+1)*64-1 : j*64] = \text{spd_reg0}[j]$.

spdc_i2c_top.v

I2C master top module. Manages the I2C physical bus: instantiates the SDA pad (IOBUF for Ultrascale FPGA, or bufif1 tristate for simulation/other targets), and wires spdc_i2c_st (the protocol FSM) and spdc_i2c_clk (the SCL clock divider). Exposes a simple handshake to spdc_ctrl: en/slv_addr as inputs, ack_err/active/done/data_read_ready/data_read/byte_cntr as outputs.

Port	Description
en	Enable: when asserted, starts a sequential I2C read of MAX_BYTE_COUNT bytes from the addressed EEPROM.
slv_addr[1:0]	I2C device address select (0=DIMM0, 1=DIMM1, 2=DIMM2). Encodes into the I2C device address byte during DEV_ADDR phase.
ack_err	Asserted if the I2C slave (EEPROM) does not ACK during address or data phases. Used by spdc_dec to detect unpopulated DIMM slots.
done	Asserted when the STOP condition has been generated — I2C transaction complete.
data_read_ready	Strobe asserted for one cycle per received byte. spdc_ctrl uses this to gate byte storage into spd_reg0/1/2.
data_read[7:0]	The most recently received I2C data byte.
byte_cntr	Running count of bytes received in the current transaction (0 to MAX_BYTE_COUNT-1).
SCL	I2C clock output, toggled by spdc_i2c_st via scl_en/scl_tick.
SDA	I2C data — bidirectional. IOBUF (Ultrascale) or bufif1 (sim) drives 0 when slv_write_en is asserted; high-Z otherwise (pulled high externally).

spdc_i2c_st.v

I2C master protocol state machine. Implements a complete DDR3 EEPROM sequential read using the standard I2C two-phase random read sequence: a random-address write (to set the internal EEPROM address pointer to 0x00), followed by a repeated-start sequential read that clocks out all MAX_BYTE_COUNT bytes. Generates SCL via scl_en/scl_tick toggling, drives SDA via slv_write_en (tristate control), and samples SDA via sda_in.

State Machine States

State	Description
IDLE	Waits for en asserted. Transitions to EN_CLK.

EN_CLK	Starts the SCL clock divider (scl_cnt_en = 1). Waits one scl_tick then proceeds.
START_RAND	Generates the START condition for the random-address write phase: pulls SDA low while SCL is high. Enables SCL toggling (scl_en = 1).
DEV_ADDR_RAND	Sends the 8-bit device address byte (write mode, R/W=0) with the 2-bit DIMM slot address embedded in bits [3:1] and [1] respectively. Controlled by slv_write_en_ combinational logic based on slv_addr and bit_cntr position.
SLV_ACK_RAND_1	Waits for the EEPROM's ACK (SDA pulled low by slave). If SDA stays high (no ACK), transitions to WAIT_2 to terminate the transaction and signals ack_err — this is the unpopulated DIMM detection mechanism.
WORD_ADDR_RAND	Sends the 8-bit word address (0x00) to set the EEPROM read pointer to the start of the SPD map.
SLV_ACK_RAND_2	Waits for EEPROM ACK after word address byte.
WAIT_RAND	One-tick pipeline delay before the repeated START.
START_SEQ	Generates the repeated START condition for the sequential read phase (START while SCL is high without a prior STOP).
DEV_ADDR_SEQ	Sends the device address byte in read mode (R/W=1 in LSB of SEQ address byte). The bit pattern driven on SDA depends on slv_addr and bit_cntr.
SLV_ACK_SEQ	Waits for EEPROM ACK after read-mode device address.
DATA_SEQ	Clocks in one byte of data from the EEPROM on SDA. Captures each bit into data_read[bit_cntr] on the rising SCL edge (when scl is high).
WAIT_1	After each received byte: if byte_cntr == MAX_BYTE_COUNT-1 (last byte), go to WAIT_2 (send NACK + STOP). Otherwise go to MST_ACK_SEQ (send ACK to continue).
MST_ACK_SEQ	Master sends ACK (pulls SDA low) to indicate it wants more bytes. Loops back to DATA_SEQ.
WAIT_2	One-tick delay before disabling SCL.
DIS_CLK	Disables SCL toggling (scl_en = 0). Waits for SCL to go high.
STOP	Generates STOP condition (SDA rises while SCL is high). Asserts done. Returns to IDLE when en deasserts.

SDA Drive Logic (slv_write_en)

slv_write_en_ is computed combinatorially, asserting 1 (drive SDA low = logical 0) during: START conditions, word address transmission, master ACK, DIS_CLK phase, and specific bit positions of the device address byte based on slv_addr. A 129-cycle pipeline register (spdc_pipeline, PIPE_NUM=129) delays slv_write_en_ to align the SDA drive with the correct I2C bit window relative to the SCL edge, compensating for the propagation delay of the SCL clock divider chain.

spdc_i2c_clk.v

Simple up-counter-based clock tick generator. Counts from 0 to $2^{\text{CNTR_W}} - 1$, then fires a single-cycle scl_tick pulse. This tick drives SCL toggling in spdc_i2c_st: each scl_tick is one half-period of the I2C SCL clock. At CNTR_W=10 and 200 MHz host clock, the full SCL period is $2 \times 1024 \times 5 \text{ ns} = 10.24 \mu\text{s} \rightarrow$ SCL frequency $\approx 97.7 \text{ kHz}$ (well within I2C standard-mode 100 kHz and fast-mode 400 kHz limits). scl_cnt_en (from spdc_i2c_st) gates the counter: when deasserted the counter holds at zero, ensuring clean SCL idle.

spdc_dec.v

The SPD decoder — the most algorithmically complex module in the project (~1200 lines). Receives raw I2C bytes via `data_ready_valid/data_in` as they stream in during the I2C read phase (parallel to `spdc_ctrl`'s register storage), uses `clock_in` (the fixed-point clock period value from `spdc_parameters.vh`) to normalize all timing values, performs fixed-point arithmetic using multiple instances of `spdc_qdiv` and `spdc_qmults`, and then streams 57 decoded DDR3 timing parameters to the DDR3MC configuration bus via `valid/data_out/address` outputs.

Two-Phase Operation

Phase 1 — Byte Capture: As bytes arrive from the I2C master (`data_ready_valid` pulses), the decoder increments a 10-bit `count_byte` and latches specific byte numbers (3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 16-22, 24-29, 33-38) into named registers (`data_byte_N`). Multi-byte fields are assembled: for example, `data_byte_22` = {`lower_nibble_b21`, `byte22`} (12 bits for `tRAS` LSBs), and `data_byte_24` = {`byte25`, `byte24`} (16 bits for `tRFC`).

Phase 2 — Decode FSM: Once `all_byte_present` is asserted (`count_byte` reaches 39, 295, or 551 — the 39th byte of each DIMM's 256-byte image), the main decode state machine runs through 18 states converting MTB/FTB byte values to nCK integer clock counts.

Decode FSM States

State	Action
Idle	Waits for <code>all_byte_present</code> or for <code>i2c_done</code> (indicating the I2C read finished for detection).
Change_clk_time	Determines the effective tCK (clock cycle time in fixed-point MTB units) from SPD bytes 10, 11, 12, 34. If byte 34 is non-zero (FTB correction present), uses <code>spdc_qmults</code> to compute $\text{FTB} \times (\text{byte9_upper}/\text{byte9_lower}) + \text{MTB} \times (\text{byte10}/\text{byte11})$, then rounds the result and stores as the system clock period for subsequent divisions.
find_tAA	Computes tAA in nCK: if byte 35 is zero, divides byte 16 by <code>clk_cycle_time</code> (simple MTB path). If byte 35 is non-zero, applies FTB correction via multipliers first. Rounds up (ceiling division: if lower bits non-zero, add 1 to upper bits).
calculate_tAA	Division completion wait state for the FTB-corrected tAA path.
find_tWR	Computes tWR: divides byte 17 by <code>clk_cycle_time</code> . Always the simple MTB path (no FTB correction byte for tWR). Stores in <code>reg_for_ddr3mc_N[12]</code> .
find_tRCD	Computes tRCD: divides byte 18 by <code>clk_cycle_time</code> . With optional FTB correction via byte 36. Stores in <code>reg_for_ddr3mc_N[13]</code> .
calculate_tRCD	FTB-corrected tRCD division completion.
find_tRRD	Computes tRRD: divides byte 19 by <code>clk_cycle_time</code> . No FTB correction. Stores in <code>reg_for_ddr3mc_N[14]</code> .
find_tRP	Computes tRP: divides byte 20 by <code>clk_cycle_time</code> . With optional FTB correction via byte 37. Stores in <code>reg_for_ddr3mc_N[15]</code> .
calculate_tRP	FTB-corrected tRP division completion.

find_tRAS	Computes tRAS: divides 12-bit data_byte_22 by clk_cycle_time using 12-bit divider. Stores in reg_for_ddr3mc_N[16].
find_tRC	Computes tRC: divides 12-bit data_byte_23 by clk_cycle_time. With optional FTB correction via byte 38. Stores in reg_for_ddr3mc_N[17].
calculate_tRC	FTB-corrected tRC division completion.
find_tRFC	Computes tRFC: divides 16-bit data_byte_24 by clk_cycle_time using 16-bit divider. Stores in reg_for_ddr3mc_N[18] and latches tRFC_nCK for use in tXPR calculation.
find_tWTR	Computes tWTR: divides byte 26 by clk_cycle_time. Stores in reg_for_ddr3mc_N[19].
find_tRTP	Computes tRTP: divides byte 27 by clk_cycle_time. Stores in reg_for_ddr3mc_N[20].
find_tFAW	Computes tFAW: divides 12-bit data_byte_29 by clk_cycle_time. Stores in reg_for_ddr3mc_N[21].
final_decode_state	Clears all_byte_present; returns to Idle. Triggers convert_state machine (see below) for JEDEC-fixed parameters.

Convert State Machine (constants not in SPD)

A parallel secondary state machine (convert_state) runs after clock_ready is set and computes timing values that are fixed by JEDEC spec rather than read from SPD: tCCD=4, tburst=4, tMRD=4 (all in nCK), and tZQINIT (640 ns), tREFI (7.8 μ s), tMOD (15 ns), tXPR (10 ns + tRFC), t500us, t200us, t10ns, t7.5ns — all converted to nCK by dividing their fixed absolute values by for_cl_cal (the computed clock cycle time in fixed-point).

DIMM Presence Detection

A dedicated dimm_check FSM monitors i2c_ack_error and current_dimm during the I2C read phase to detect which DIMM slots are physically populated. If a DIMM is absent, its I2C transaction produces an ACK error at byte_cntr==0; the decoder then skips that DIMM's slot by setting inc_count, which jumps count_byte forward by 256 to skip to the next DIMM's byte range.

Output Streaming

After decode is complete, the decoder streams 57 entries to the DDR3MC config interface keyed by byte_cmp (a 9-bit counter). Address 0xFFFF is a start/end sentinel. Addresses 0x0000..0x0002 report DIMM connectivity (which of the 3 slots are populated). Addresses 0x0003..0x0038 carry the per-DIMM and cross-DIMM-max timing values. For multi-DIMM configurations, the decoder applies compare_3_value() or compare_2_value() functions to select the worst-case (largest) timing value across all connected DIMMs, ensuring DDR3MC timing margins are satisfied for the most conservative DIMM in the system.

spdc_dec_beta.v

An incomplete, stub/beta version of the SPD decoder with a different architectural approach. Rather than the byte-by-byte streaming decode of spdc_dec, this module reads the fully-assembled reg0/reg1/reg2 register banks after the fact and uses combinational byte-slice extraction (assign dimm0_tAA = reg0[(BYTE_tAA*8)-1 : (BYTE_tAA*8)-7]) to parse all SPD fields. The FSM body is incomplete (syntactically broken — the case statement references IDLE but defines it with begin without proper case end), and the module appears to

have been an experimental rewrite that was abandoned before `spdc_dec` was completed. It is present in the source tree but is not instantiated anywhere in the active RTL.

spdc_pipeline.v

Generic parameterized shift register pipeline. Parameters: `PIPE_NUM` (number of pipeline stages, minimum 1) and `DATA_WIDTH` (bit width of each stage). Instantiated once in `spdc_i2c_st` with `PIPE_NUM=129` and `DATA_WIDTH=1` to delay the combinatorial `slv_write_en` signal by 129 clock cycles before it reaches the IOBUF T pin, aligning the SDA bus drive timing with the correct I2C bit window. The synthesis preserve attribute prevents register retiming or optimization from collapsing the chain.

spdc_qadd.v

Fixed-point signed addition module, parameterized by `Q` (fractional bits) and `N` (total width). Implements sign-magnitude addition: checks the MSB (sign bit) of both operands and performs either addition or subtraction of the magnitude portions accordingly. Purely combinational (no clock). Instantiated twice in `spdc_dec` as `spdc_qadd_inst1` (`Q=5`, `N=13`, adds FTB-corrected `mult1_result` + MTB-shifted `mult2_result` for `tCK/tAA/tRCD/tRP` paths) and `spdc_qadd_inst2` (`Q=5`, `N=13`, same for the `tRC` FTB path).

spdc_qdiv.v

Fixed-point sequential divider, parameterized by `Q` (fractional bits) and `N` (bit width). Implements non-restoring division over `N+Q` clock cycles. `i_start` triggers the operation; `o_complete` pulses when done; `o_quotient_out` is the result in `Q.N-Q` fixed-point. `o_overflow` indicates the result exceeds the representable range. Uses a working quotient register (`2N+Q-3` bits) and shift-subtract algorithm.

Instantiated 7 times in `spdc_dec` with different `Q/N` configurations: `Q3/N24` for `nCK` conversions of large time values (`tZQINIT`, `tREFI`, etc.), `Q3/N9` for `CL` calculation, `Q3/N8` for 8-bit MTB path divisions (`tAA`, `tWR`, `tRCD`, `tRRD`, `tRP`, `tWTR`, `tRTP`), `Q3/N8` again for FTB ratio calculation, `Q3/N12` for 12-bit values (`tRAS`, `tRC`, `tFAW`), `Q3/N16` for 16-bit `tRFC`, and `Q5/N13` for FTB fine-timebase ratio.

spdc_qmults.v

Fixed-point signed sequential multiplier, parameterized by `Q` (fractional bits) and `N` (total width). Implements a shift-add multiplication over `N` clock cycles. `i_start` triggers; `o_complete` pulses when done; `o_result_out` is the lower `N` bits of the `2N-1`-bit product (the `Q` fractional bits worth of precision). Instantiated 4 times in `spdc_dec`: instances 1 and 2 (`Q=5`, `N=13` and `Q=3`, `N=8`) for the `FTB × (FTB_reg)` and `MTB × MTB_reg` multiplications in the `tCK/tAA/tRCD/tRP/tRC` paths; instances 3 and 4 (`Q=5`, `N=13` and `Q=3`, `N=8`) for the same operations in the separate `tRC` FTB-correction path.

spdc_testDrv.sv

Minimal test stimulus driver. Asserts `dec_en` immediately on reset deassertion (`IDLE → EN` in one cycle) and holds it until it detects the falling edge of `dec_data_valid` (indicating the decoder has finished streaming its last output entry). Once the falling edge is detected, transitions to `DONE` and holds. Under `SYNTHS`, a Xilinx VIO probe would be used to override `dec_en` for interactive hardware control, though that path is not currently wired in `spdc_wrap.sv`.

sim/

tb_spdc_top.sv

Top-level testbench for spdc_top. Generates a 200 MHz clock (CLK_PER=5ns), drives reset, and executes a test sequence: after 10 μ s from reset deassertion, calls decRead() (which asserts dec_en) and then spdRegCompare(0) (which checks reg0 contents against expected SPD byte values). The testbench instantiates spdc_top as the DUT, and connects it to the sim-mode I2C BFM (24AA02.v or 24AA64.v) through the i2c_SDA/SCL wires.

sim/spd_eeprom.sv

SPD EEPROM simulation wrapper. Instantiates one or more Microchip I2C EEPROM behavioral models (24AA64.v) and wires their address pins from the i2c_sa input, modeling a real DDR3 DIMM's SPD EEPROM device on the I2C bus for simulation. The EEPROM model responds to the correct I2C device address and returns pre-loaded SPD byte contents, allowing full end-to-end verification of the I2C master and decoder.

sim/24AA64.v

Third-party behavioral Verilog model of the Microchip 24AA64 64 Kbit (8KB) I2C serial EEPROM — the primary SPD EEPROM model used in simulation. Implements the complete I2C slave protocol: START/STOP detection, device address matching, word address reception, and sequential data output from an internal 8192-byte memory array. The array can be initialized via \$readmemh for test-specific SPD content.

sim/24AA02.v

Third-party behavioral Verilog model of the Microchip 24AA02 2 Kbit (256 byte) I2C serial EEPROM — the smaller variant, exactly matching the 256-byte DDR3 SPD page size. Used in some simulation configurations (referenced in spd_eeprom.sv comments) as an alternative to 24AA64. Same behavioral protocol as 24AA64 but limited to 256 bytes, which matches MAX_BYTE_COUNT perfectly.

eeprom.txt

Plain-text hex initialization file. Contains 256 bytes of SPD data in text hex format, loaded into the 24AA02/24AA64 behavioral model's memory array via \$readmemh. The values represent a real DDR3 SODIMM's SPD image used during functional regression simulation of the SPDC decode path.

const/ — FPGA Implementation Constraints

Xilinx Design Constraints (XDC) files for two target board/FPGA combinations (F1A/fpgaA and F4A/fpgaBD), each in a single-rank (x1) and triple-rank (x3) memory configuration variant. These assign physical FPGA package pins to the I2C bus signals (SDA, SCL, SA) and provide clock/reset timing constraints for synthesis and implementation.

File	Purpose
spdc_io_F1A_fpgaA_x1.xdc	I2C pin assignments for FPGA A (F1A board), single-rank memory configuration
spdc_io_F1A_fpgaA_x3.xdc	I2C pin assignments for FPGA A (F1A board), triple-rank memory configuration
spdc_io_F4A_fpgaBD_x1.xdc	I2C pin assignments for FPGA B/D (F4A board), single-rank configuration
spdc_io_F4A_fpgaBD_x3.xdc	I2C pin assignments for FPGA B/D (F4A board), triple-rank configuration
spdc_wrap_F1A_fpgaA_x1.xdc	Clock source, reset, and top-level timing constraints for spdc_wrap on FPGA A (x1)
spdc_wrap_F1A_fpgaA_x3.xdc	Clock source, reset, and top-level timing constraints for spdc_wrap on FPGA A (x3)
spdc_wrap_F4A_fpgaBD_x1.xdc	Clock and timing constraints for FPGA B/D (x1)
spdc_wrap_F4A_fpgaBD_x3.xdc	Clock and timing constraints for FPGA B/D (x3)

scripts/ — Build & Simulation Scripts

File	Purpose
spdc_envsetup.tcl	Vivado Tcl environment setup: defines helper procedures for resolving project and source directory paths. Sourced first by other Tcl scripts to establish the build context.
spdc_define.tcl	Vivado compile-time define settings: sets synthesis and simulation Verilog defines (e.g. SYNTHS for synthesis, SIM for simulation), selects the active filesset, and configures XSIM runtime (simulation duration) and logging options.
spdc_ip.tcl	Vivado IP catalog script: creates and configures the MMCM/PLL clock generator IP (clkgen) used by spdc_wrap in synthesis mode. Generates a 200 MHz output from a differential reference input. Can be sourced in Vivado to regenerate the IP if constraints change.

runVCS

VCS (Synopsys) compile and simulation run script for standalone Linux-based simulation outside of Vivado. Compiles all RTL files with +define+SIM and -timescale=1ns/1ns flags, then runs the top-level testbench (tb_spdc_top) with full toggle/activity coverage logging. Used during development to quickly iterate on the decoder algorithm without the Vivado GUI.

4. End-to-End Data Flow

4.1 SPD Read Phase (I2C → Register Banks)

1. `spdc_wrap` deasserts `reset`; `spdc_top` immediately enters `SPD_WRITE_CMD` and drives `CMD_SPD_WR` to `spdc_ctrl`.
2. `spdc_ctrl` enters `SPD_WR_REG0_EN`: sets `i2c_slv_addr=0` and `i2c_en=1`, triggering `spdc_i2c_top` for DIMM0.
3. `spdc_i2c_st` executes the full I2C 2-phase random-read: `START` → write device address → write word address `0x00` → repeated `START` → read device address → stream 256 bytes → `STOP`. Each received byte triggers `data_read_ready` for one cycle.
4. On each `data_read_ready` pulse, `spdc_ctrl` shifts the byte into `spd_reg0[byte_cntr>>3]` and `spdc_dec` latches it into `data_byte_N` (if it matches a decoded byte position). `byte_cntr` increments.
5. When `i2c_done` asserts (`STOP` state reached), `spdc_ctrl` moves to `SPD_WR_REG0_DONE`, waits for `i2c_done` to deassert, then advances to `SPD_WR_REG1_EN` with `i2c_slv_addr=1` for DIMM1. Same sequence repeats, storing into `spd_reg1`.
6. After DIMM2 (`i2c_slv_addr=2`, `spd_reg2`), `spdc_ctrl` pulses `done`. `spdc_top` exits `SPD_WRITE_CMD_WAIT` and enters `WAIT`.

4.2 Decode Phase (Register Banks → nCK Parameters)

1. `spdc_testDrv` immediately asserts `dec_en`; `spdc_top` transitions to `DEC_READ_CMD`, issues `CMD_DEC_RD` to `spdc_ctrl`.
2. `spdc_ctrl` enters `DEC_RD_EN` and pulses `dec_rd_en` to `spdc_dec` for one cycle. Then waits for `dec_rd_done`.
3. `spdc_dec` checks the flag signal (`count_byte==768 = 3 DIMMs × 256 bytes done`). With `spd_wr_en` asserted (from `dec_rd_en`), the output streaming state machine begins at `byte_cmp=0`.
4. `spdc_dec` iterates `byte_cmp` from 0 to 58: for each step it presents a `data_out/address/valid` entry. The first entry (`address=0xFFFF`, `data=1`) is the start sentinel. Addresses `0x0000-0x0002` report DIMM0/1/2 connection status. Addresses `0x0003-0x0038` carry the 54 timing parameter values (per-DIMM geometry values followed by worst-case timing values across all DIMMs). The final entry (`address=0xFFFF`, `data=0`) is the end sentinel.
5. After 58 steps, `valid` goes low and `decode_done` is asserted → `dec_rd_done` propagates back to `spdc_ctrl` → `done` pulses to `spdc_top` → `spdc_top` returns to `WAIT`.

5. Module Summary

Module / File	Location	Function
spdc_parameters.vh	include/	Shared parameters: clock period, command encodings, widths, fixed-point clock value
spdc_wrap.sv	rtl/	FPGA harness: MMCM clock, test driver, DUT instantiation
spdc_top.v	rtl/	Top-level FSM: sequences SPD-write, wait, and decode-read phases
spdc_ctrl.v	rtl/	Command dispatcher: I2C sequencing, SPD register storage, decoder trigger
spdc_i2c_top.v	rtl/	I2C master top: SDA pad, wires FSM and clock divider
spdc_i2c_st.v	rtl/	I2C master protocol FSM: full 2-phase random-read with ACK detection
spdc_i2c_clk.v	rtl/	SCL clock divider: counter $\rightarrow$ scl_tick pulse at $1/2^{\text{CNTR_W}}$ rate
spdc_dec.v	rtl/	SPD decoder: byte capture, fixed-point decode FSM, 57-entry nCK output stream
spdc_dec_beta.v	rtl/	Incomplete/abandoned alternate decoder (not instantiated)
spdc_pipeline.v	rtl/	Parameterized shift-register pipeline; used for SDA timing alignment (PIPE_NUM=129)
spdc_qadd.v	rtl/	Fixed-point signed adder (Q/N parameterized, combinational)
spdc_qdiv.v	rtl/	Fixed-point sequential divider (Q/N parameterized, N+Q clock cycles)
spdc_qmults.v	rtl/	Fixed-point signed sequential multiplier (Q/N parameterized)
spdc_testDrv.sv	rtl/	Test stimulus: asserts dec_en immediately, holds until decode complete
tb_spdc_top.sv	sim/	Testbench: clock/reset gen, calls decRead() and spdRegCompare()
spd_eeprom.sv	sim/	EEPROM wrapper: instantiates 24AA64 model with I2C address pins
24AA64.v	sim/	Behavioral Microchip 24AA64 64 Kbit I2C EEPROM model
24AA02.v	sim/	Behavioral Microchip 24AA02 2 Kbit I2C EEPROM model (256 bytes)
eeprom.txt	sim/	Hex SPD data initialization file for EEPROM behavioral model
spdc_io_*.xdc	const/	I2C bus pin assignments for F1A/F4A boards, x1/x3 variants
spdc_wrap_*.xdc	const/	Clock and timing constraints for synthesis, F1A/F4A boards
spdc_envsetup.tcl	scripts/	Vivado environment path helpers
spdc_define.tcl	scripts/	Vivado compile-time defines and simulation settings

spdc_ip.tcl	scripts/	Vivado MMCM IP core generation script
runvcs	root/	Standalone VCS simulation build and run script